Reliable SQL Reports

Count requests. Summarize histories. Select the latest event.

Day 1: report counts and common table expressions
Day 2: ROW_NUMBER and one latest event per request

Start with the same three tables

Table	One row	Rows
users	One person	8
tickets	One request; optional assignee	12
ticket_events	One recorded event on a request	21

Open the Week 4 notebook: introduction -> setup -> inspect the tables.
Active tickets: 7. Unassigned active tickets: 1004 and 1009.

One request can have three history rows

event_id	ticket_id	event_type	new_status
5005	1003	created	open
5006	1003	status_changed	in_progress
5007	1003	status_changed	resolved

Three events describe one ticket. Decide which fact the report needs.

A join produces matching pairs

```
SELECT t.ticket_id, t.subject, t.status, e.event_id
FROM metro_support.tickets AS t
JOIN metro_support.ticket_events AS e
  ON e.ticket_id = t.ticket_id
WHERE t.ticket_id = 1003
ORDER BY e.event_at, e.event_id;
```

Result: 3 ticket-event pairs, not 3 distinct requests.

A plausible request count can count events instead

```
SELECT t.category, count(*) AS requests
FROM metro_support.tickets AS t
JOIN metro_support.ticket_events AS e
  ON e.ticket_id = t.ticket_id
GROUP BY t.category ORDER BY t.category;
```

Category	Misnamed requests	Actual tickets
water	5 event pairs	2 requests
All categories	21 event pairs	12 requests

Define each count before writing the expression

```
SELECT count(*) AS tickets,
       count(assignee_id) AS assigned_tickets,
       count(DISTINCT assignee_id) AS different_assignees
FROM metro_support.tickets;

-- One conditional count, without removing other rows:
-- count(*) FILTER (WHERE status = 'resolved')
```

12 tickets / 10 assigned tickets / 2 distinct assignees

One category row can contain two measures

```
SELECT category, count(*) AS total_tickets,
       count(*) FILTER (WHERE status = 'resolved')
         AS resolved_tickets
FROM metro_support.tickets
GROUP BY category ORDER BY category;
```

category	total_tickets	resolved_tickets
parks	2	0
sanitation	3	1
streetlight	3	1
transportation	2	0
water	2	2

A CTE names an intermediate result

```
WITH event_counts AS (
  SELECT ticket_id, count(*) AS event_count
  FROM metro_support.ticket_events
  GROUP BY ticket_id
)
SELECT * FROM event_counts ORDER BY ticket_id;
```

Intermediate result: at most one row per ticket_id.
1003 has 3 events; 1004 has 1.

Summarize first; then join once per ticket

```
CREATE OR REPLACE VIEW metro_support.ticket_event_counts AS
WITH event_counts AS (
  SELECT ticket_id, count(*) AS event_count
  FROM metro_support.ticket_events GROUP BY ticket_id
)
SELECT t.ticket_id, t.subject,
       COALESCE(e.event_count, 0) AS event_count
FROM metro_support.tickets AS t
LEFT JOIN event_counts AS e ON e.ticket_id = t.ticket_id;
```

One output row per ticket, including a ticket with no recorded event.

Check population, uniqueness, and a known record

```
SELECT count(*) AS report_rows,
       count(DISTINCT ticket_id) AS distinct_tickets,
       sum(event_count) AS recorded_events
FROM metro_support.ticket_event_counts;

SELECT ticket_id, event_count
FROM metro_support.ticket_event_counts
WHERE ticket_id IN (1003, 1004) ORDER BY ticket_id;
```

12 rows / 12 ticket IDs / 21 events. Known counts: 1003 -> 3; 1004 -> 1.

Zero, NULL, and a missing row make different claims

Observed result	Meaning in this report
event_count = 0	Included ticket; no event rows found
assignee_id IS NULL	Included ticket; no person assigned
Ticket absent from output	Ticket was not included in this result

Keep the report population separate from the conditions inside each count.

Lab 1: a category workload report

One row per category:
all tickets / active tickets / high-or-urgent active tickets

Keep 5 categories. Check totals of 12 / 7 / 2.
Independently list the 2 high-or-urgent active ticket IDs.

Brightspace: week_04_category_report.sql
One SQL file, with brief explanations as comments.

One latest recorded event per ticket

Day 2
Rank history rows, select one, and preserve the request population.

Open the same Week 4 notebook. Start at the Day 2 heading after setup.

MAX finds a value, not its entire row

```
SELECT ticket_id, max(event_at) AS latest_time
FROM metro_support.ticket_events
GROUP BY ticket_id;
```

This finds a timestamp. It does not also select the note or event type from that row.

MAX(event_at) and MAX(note) can come from different rows.

ROW_NUMBER labels rows within each ticket

```
SELECT ticket_id, event_id, event_type, event_at,
       ROW_NUMBER() OVER (
         PARTITION BY ticket_id
         ORDER BY event_at DESC, event_id DESC
       ) AS rn
FROM metro_support.ticket_events
ORDER BY ticket_id, rn;
```

PARTITION BY: restart numbering per ticket.
ORDER BY inside OVER: decide which event receives 1.

Ranking keeps the original event rows

ticket_id	event_id	position rn	Selected later
1003	5007	1	Yes
1003	5006	2	No
1003	5005	3	No
1004	5008	1	Yes

The ranking query still returns all 21 event rows.

Filter the ranking in an outer query

```
WITH ranked_events AS (
  SELECT ticket_id, event_id, event_type, event_at,
         ROW_NUMBER() OVER (
           PARTITION BY ticket_id
           ORDER BY event_at DESC, event_id DESC
         ) AS rn
  FROM metro_support.ticket_events
)
SELECT ticket_id, event_id, event_type, event_at
FROM ranked_events WHERE rn = 1
ORDER BY ticket_id;
```

First number the rows. Then keep position 1. The stored history is unchanged.

Attach the selected event to each request

```
WITH ranked_events AS (
  SELECT ticket_id, event_id, event_type, event_at,
         ROW_NUMBER() OVER (PARTITION BY ticket_id
           ORDER BY event_at DESC, event_id DESC) AS rn
  FROM metro_support.ticket_events
)
SELECT t.ticket_id, t.status, e.event_id, e.event_type, e.event_at
FROM metro_support.tickets AS t
LEFT JOIN ranked_events AS e
  ON e.ticket_id = t.ticket_id AND e.rn = 1
ORDER BY t.ticket_id;
```

The rn = 1 condition belongs in ON when unmatched tickets must remain.

Equal timestamps need an explicit tie rule

Illustrative event	Recorded time	Tie-break result
7001	2026-02-15 12:00:00+00	Position 2
7002	2026-02-15 12:00:00+00	Position 1

```
ORDER BY event_at DESC, event_id DESC
Deterministic selection is not proof of real-world arrival
order.
```

A missing history row must not hide the ticket

Ticket case	Result from LEFT JOIN	Incorrect shortcut
Recorded events exist	Ticket plus its selected event	Join every event and repeat the ticket
No event recorded	Ticket plus NULL event fields	Filter the unmatched ticket away

Preserve the request; display the absence of history honestly.

A view exposes the report without copying the data

```
CREATE OR REPLACE VIEW metro_support.ranked_ticket_events AS
SELECT ticket_id, event_id, event_type, event_at,
       ROW_NUMBER() OVER (PARTITION BY ticket_id
         ORDER BY event_at DESC, event_id DESC) AS rn
FROM metro_support.ticket_events;

SELECT * FROM metro_support.ranked_ticket_events
WHERE rn = 1 ORDER BY ticket_id;
```

A view stores the query definition. Its next read can reflect newly committed events.

Test the selected row, not only the row count

```
SELECT ticket_id, count(*)
FROM metro_support.active_ticket_latest_event
GROUP BY ticket_id
HAVING count(*) > 1;

-- Expected after the lab view exists: no duplicate-ID rows.
-- Also check the population and selected event identifiers.
```

Check	Expected on the starting fixture
Active report population	7 distinct tickets; includes 1004 and 1009
Selected events for those two	1004 -> 5008; 1009 -> 5016

Lab 2: the active queue with its latest event

Create metro_support.active_ticket_latest_event
using the worked ROW_NUMBER + LEFT JOIN pattern.

Keep 7 active ticket IDs, including 1004 and 1009.
Check their selected events and check for duplicate tickets.

Brightspace: week_04_latest_event.sql
One SQL file with short explanatory comments.

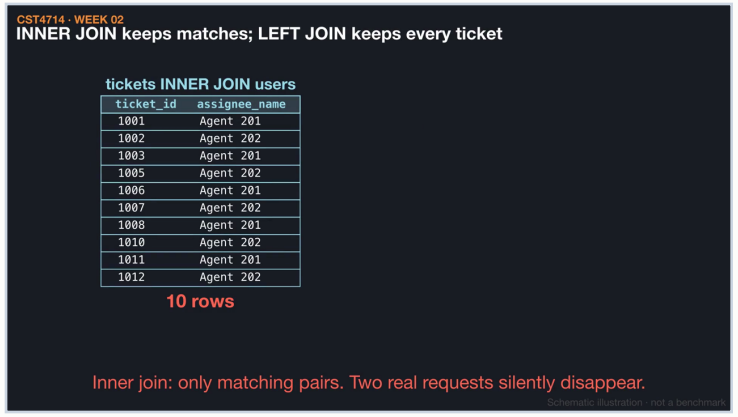
A correct query still reads a changing database

Query correctness: which rows should this statement return?

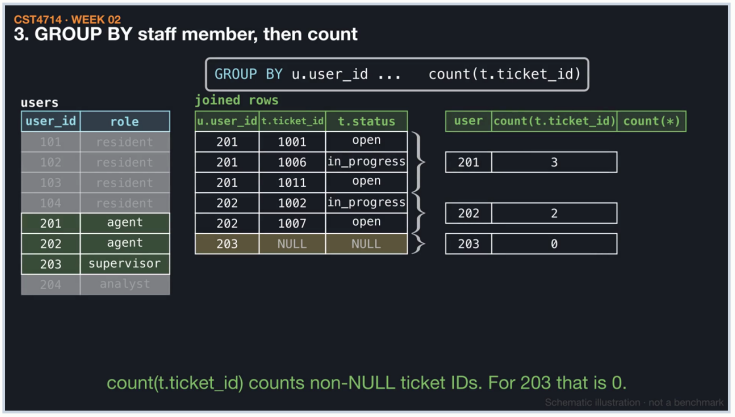
Transaction behavior: which committed changes can it see?

Next: transactions, visibility, and blocking.
Keep your SQL; clean up only the notebook database created for this exercise.

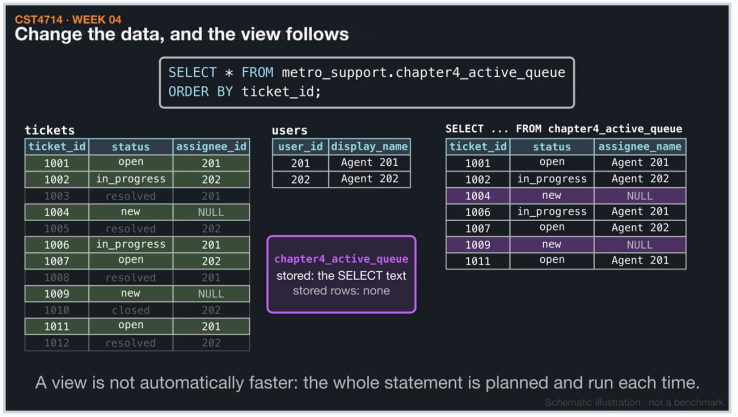
This week's videos



What does one row mean after a join? (1:16)



Counting workload, including zero (1:28)



A view stores a query, not rows (1:14)

Click a thumbnail to play on GitHub. Silent, with captions and .srt subtitles; diagrams are schematic.